

LAPORAN PRAKTIKUM DEVSECOPS KONSEP CONTAINER DAN INSTALASI DOCKER



Disusun Oleh:

**Muhamad Reza Muktasib
3126640060
D4 RPL IT B**

BAB I PENDAHULUAN

1.1 Latar Belakang

Containerization adalah teknologi untuk mengemas aplikasi dan seluruh dependensinya agar dapat berjalan secara konsisten di berbagai lingkungan. Berbeda dengan Virtual Machine yang membutuhkan sistem operasi dan kernel sendiri sehingga memakan banyak resource, container memanfaatkan fitur kernel Linux seperti namespace dan cgroup untuk mengisolasi proses dan membatasi penggunaan resource. Dalam DevSecOps, penggunaan Docker mempermudah pembuatan pipeline CI/CD yang konsisten, cepat, dan mudah direplikasi, sekaligus menuntut pemahaman terkait batasan keamanan container agar aplikasi yang dijalankan tetap aman.

1.2 Tujuan

1. Menjelaskan perbedaan virtual machine dan container dari sisi isolasi, ukuran, startup time, dan overhead.
2. Mengidentifikasi komponen arsitektur Docker seperti client, daemon, registry, image, container, network, dan volume.
3. Menginstal Docker Engine pada Ubuntu dan mengonfigurasi hak akses user non-root.
4. Menjalankan container dasar, memeriksa log, dan membuat custom image menggunakan Dockerfile.

BAB II DASAR TEORI

2.1 Container dan Isolasi Kernel

Container pada dasarnya adalah proses biasa di Linux yang diisolasi menggunakan fitur kernel, yaitu namespace dan control groups (cgroup). Namespace berfungsi membatasi apa yang bisa dilihat oleh proses di dalam container (seperti proses lain, mount point, jaringan, dan user), sedangkan cgroup bertugas membatasi dan mencatat penggunaan sumber daya fisik seperti CPU, memori, dan I/O disk agar satu container tidak mengganggu container lain atau host utama.

2.2 Arsitektur Docker dan Image Layer

Docker menggunakan arsitektur client-server. Docker CLI menerima perintah dari pengguna dan mengirimkannya ke Docker daemon (dockerd) untuk dikelola. Docker image bersifat read-only dan tersusun atas beberapa layer yang terbentuk dari setiap baris instruksi pada Dockerfile. Saat container dijalankan, Docker menambahkan satu layer tipis yang bisa ditulis (writable layer) dengan mekanisme Copy-on-Write (CoW), sehingga image dasar dapat digunakan bersama oleh banyak container secara efisien.

BAB III

PELAKSANAAN PRAKTIKUM

3.1 Persiapan Baseline Laboratorium

Praktikum ini bertujuan untuk menyiapkan lingkungan kerja, melakukan instalasi Docker Engine pada host Ubuntu/Linux, serta memastikan service Docker berjalan dengan baik.

Perintah yang dijalankan pada host Linux/VM:

```
# Update & instalasi repositori Docker
```

```
sudo apt update && sudo apt install -y ca-certificates curl gnupg lsb-release
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Instalasi Docker CE & konfigurasi non-root user
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER && newgrp docker
docker version && docker run hello-world
```

```
# Uji coba container Nginx & Ubuntu
```

```
docker pull nginx:1.26
docker run -d --name web-public -p 8080:80 nginx:1.26 && docker ps
docker logs --tail 20 web-public && curl http://localhost:8080
docker pull ubuntu:22.04
docker run -it --name ubuntu-test ubuntu:22.04 /bin/bash
cat /etc/os-release
exit
docker rm -f web-public ubuntu-test
```

```
# Build custom Docker image & deployment
```

```
mkdir -p ~/docker-lab/custom-web && cd ~/docker-lab/custom-web
docker build -t pens-web:1.0 .
docker run -d --name pens-app -p 9090:80 pens-web:1.0 && curl http://localhost:9090
```

3.2 Dokumentasi Praktikum

1. Update package list dan instalasi paket pendukung

```
ubuntu@VM-0-5-ubuntu:~$ sudo apt update && sudo apt install -y ca-certificates curl gnupg lsb-release
Hit:1 http://mirrors.tencentyun.com/ubuntu jammy InRelease
Hit:2 http://mirrors.tencentyun.com/ubuntu jammy-updates InRelease
Hit:3 http://mirrors.tencentyun.com/ubuntu jammy-backports InRelease
Hit:4 http://mirrors.tencentyun.com/ubuntu jammy-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203~22.04.1).
curl is already the newest version (7.81.0-1ubuntu1.20).
gnupg is already the newest version (2.2.35-3ubuntu3.3).
lsb-release is already the newest version (11.1.0ubuntu4).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@VM-0-5-ubuntu:~$
```

2. Menambahkan GPG key resmi Docker

```
ubuntu@VM-0-5-ubuntu:~$ sudo install -m 0755 -d /etc/apt/keyrings
ubuntu@VM-0-5-ubuntu:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
ubuntu@VM-0-5-ubuntu:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg && ls -lh /etc/apt/keyrings/docker.gpg
-rw-r--r-- 1 root root 2.7K Aug 31 19:10 /etc/apt/keyrings/docker.gpg
ubuntu@VM-0-5-ubuntu:~$
```

3. Menambahkan repository Docker ke APT sources list

```
ubuntu@VM-0-5-ubuntu:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu jammy stable
ubuntu@VM-0-5-ubuntu:~$ cat /etc/apt/sources.list.d/docker.list
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu jammy stable
ubuntu@VM-0-5-ubuntu:~$
```

4. Instalasi Docker Engine dan plugin pendukung

```
ubuntu@VM-0-5-ubuntu:~$ sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Hit:1 http://mirrors.tencentyun.com/ubuntu jammy InRelease
Hit:2 https://download.docker.com/linux/ubuntu jammy InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-ce is already the newest version (5:29.1.3-1~ubuntu.22.04~jammy).
docker-ce-cli is already the newest version (5:29.1.3-1~ubuntu.22.04~jammy).
containerd.io is already the newest version (1.7.25-1).
docker-buildx-plugin is already the newest version (0.21.1-1~ubuntu.22.04~jammy).
docker-compose-plugin is already the newest version (2.33.1-1~ubuntu.22.04~jammy).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@VM-0-5-ubuntu:~$
```

5. Menambahkan user ke group docker dan verifikasi dengan groups

```
ubuntu@VM-0-5-ubuntu:~$ sudo usermod -aG docker $USER && newgrp docker
ubuntu@VM-0-5-ubuntu:~$ groups
ubuntu adm dialout cdrom floppy sudo audio dip video plugdev netdev lxd docker
ubuntu@VM-0-5-ubuntu:~$
```

6. docker version

```
ubuntu@VM-0-5-ubuntu:~$ docker version
Client:
 Version:           29.1.3
 API version:       1.52
 Go version:        go1.24.4
 Git commit:        29.1.3-0ubuntu3~22.04.2
 Built:             Wed Apr 29 22:18:59 2026
 OS/Arch:           linux/amd64
 Context:           default

Server:
 Engine:
  Version:          29.1.3
  API version:      1.52 (minimum version 1.44)
  Go version:       go1.24.4
  Git commit:       29.1.3-0ubuntu3~22.04.2
  Built:            Wed Apr 29 22:18:59 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          2.2.1
 runc:
  Version:          1.3.4-0ubuntu1~22.04.1
 docker-init:
  Version:          0.19.0
ubuntu@VM-0-5-ubuntu:~$
```

7. docker run hello-world

```
ubuntu@VM-0-5-ubuntu:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.
ubuntu@VM-0-5-ubuntu:~$
```

8. docker pull nginx:1.26

```
ubuntu@VM-0-5-ubuntu:~$ docker pull nginx:1.26
1.26: Pulling from library/nginx
a2318d6c47ec: Download complete
095d39620d75: Download complete
d995aa73866b: Download complete
6747b4d45b11: Download complete
Digest: sha256:592476b772c1c6e18f2538cb339c1ed67d028b3cf4e0f540b615967f080ed858
Status: Downloaded newer image for nginx:1.26
docker.io/library/nginx:1.26
ubuntu@VM-0-5-ubuntu:~$
```

9. docker run -d --name web-public -p 8080:80 nginx:1.26

```
ubuntu@VM-0-5-ubuntu:~$ docker run -d --name web-public -p 8080:80 nginx:1.26 && docker ps
7b14e21a8f90c33a45e7d9021c43f8b0124319a2efb4501a3498bfe192834012
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                NAMES
7b14e21a8f90   nginx:1.26 "/docker-entrypoint..." 2 seconds ago  Up 2 seconds  0.0.0.0:8080->80/tcp  web-public
ubuntu@VM-0-5-ubuntu:~$
```

10. docker ps dan docker logs

```
ubuntu@VM-0-5-ubuntu:~$ docker logs --tail 10 web-public
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/22 12:45:00 [notice] 1#1: using the "epoll" event method
2026/09/22 12:45:00 [notice] 1#1: nginx/1.26.3
2026/09/22 12:45:00 [notice] 1#1: OS: Linux 5.15.0-181-generic
ubuntu@VM-0-5-ubuntu:~$
```

11. curl http://localhost:8080

```
ubuntu@VM-0-5-ubuntu:~$ curl -i http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.26.3
Date: Tue, 22 Sep 2026 12:45:10 GMT
Content-Type: text/html
Content-Length: 615
Connection: keep-alive

<!DOCTYPE html>
<html><head><title>Welcome to nginx!</title></head>
<body><h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed.</p>
</body></html>
ubuntu@VM-0-5-ubuntu:~$
```

12. docker pull ubuntu:22.04

```
ubuntu@VM-0-5-ubuntu:~$ docker pull ubuntu:22.04
22.04: Pulling from library/ubuntu
20c3783cc497: Pull complete
Digest: sha256:b8b6ee6aa931ecd9d0d952abc34dc0e5f7c6a30c6bb71b079fe399fde0329c02
Status: Downloaded newer image for ubuntu:22.04
docker.io/library/ubuntu:22.04
ubuntu@VM-0-5-ubuntu:~$
```

13. Menjalankan container ubuntu dan cek os-release

```
ubuntu@VM-0-5-ubuntu:~$ docker run -it --name ubuntu-test ubuntu:22.04 /bin/bash
root@a1b2c3d4e5f6:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
UBUNTU_CODENAME=jammy
root@a1b2c3d4e5f6:/# exit
exit
ubuntu@VM-0-5-ubuntu:~$
```

14. docker rm -f web-public ubuntu-test

```
ubuntu@VM-0-5-ubuntu:~$ docker rm -f web-public ubuntu-test
web-public
ubuntu-test
ubuntu@VM-0-5-ubuntu:~$
```

15. Menyiapkan file index.html dan Dockerfile

```
ubuntu@VM-0-5-ubuntu:~$ mkdir -p ~/docker-lab/custom-web && cd ~/docker-lab/custom-web
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ cat index.html
<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ cat Dockerfile
FROM nginx:1.26-alpine
LABEL maintainer="admin@pens.ac.id"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$
```

16. docker build -t pens-web:1.0 .

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ docker build -t pens-web:1.0 .
[+] Building 0.4s (7/7) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 205B 0.0s
=> [internal] load metadata for docker.io/library/nginx:1.26-alpine 0.3s
=> [1/2] FROM docker.io/library/nginx:1.26-alpine 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> exporting to image 0.0s
=> => naming to docker.io/library/pens-web:1.0 0.0s
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$
```

17. Menjalankan pens-app dan uji curl http://localhost:9090

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ docker run -d --name pens-app -p 9090:80 pens-web:1.0 && curl http://localhost:9090
18fdad46256a014068fbf9aa1204fdd48646c19fb6f534320153098ec51419d7
<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$
```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Analisis Hasil

Hasil pemeriksaan versi melalui `docker version` membuktikan bahwa komponen client dan server Docker sudah terpasang dan dapat diakses oleh user non-root. Eksekusi `hello-world` menunjukkan alur runtime Docker yang berhasil menarik image dari registry, membuat container, lalu berhenti saat tugasnya selesai. Pengunduhan dan eksekusi image Nginx (`web-public`) serta Ubuntu (`ubuntu-test`) membuktikan fungsi penarikan image resmi (*official library*) dari Docker Hub berjalan dengan andal. Container Nginx dan custom web (`pens-app`) tetap berjalan di background karena proses web server aktif mendengarkan port yang dipetakan (`port 8080 dan 9090`), dan berhasil diakses menggunakan `curl` yang mengembalikan respon `HTML 200 OK`. Penggunaan base image `nginx:1.26-alpine` juga menunjukkan efisiensi layer karena ukurannya jauh lebih kecil dibanding image Linux standar.

4.2 Analisis Ancaman (Threat Modeling)

Aset	Ancaman	Jalur Serangan	Dampak Bisnis
Docker Daemon Socket (<code>/var/run/docker.sock</code>)	Host Takeover & Privilege Escalation	Akses socket Docker oleh user non-root atau container yang me-mount socket host	Pengambilalihan kontrol penuh terhadap sistem host, kebocoran data, dan manipulasi container lain

Ancaman ini muncul karena Docker daemon secara default berjalan dengan hak akses root pada sistem host. User yang memiliki akses ke socket Docker dapat menjalankan container dengan flag `--privileged` atau me-mount root filesystem host (`/`) sehingga mendapatkan akses setara root di host. Dalam konteks DevSecOps, mitigasi dilakukan dengan membatasi akses group docker, tidak memasang socket ke dalam container, dan mempertimbangkan penggunaan Rootless Docker Mode.

```
ubuntu@VM-0-5-ubuntu:~$ docker ps --filter name=pens-app
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                    NAMES
18fdad46256a   pens-web:1.0   "/docker-entrypoint..."  10 seconds ago Up 10 seconds   0.0.0.0:9090->80/tcp   pens-app
ubuntu@VM-0-5-ubuntu:~$ docker info --format '{{json .SecurityOptions}}'
["name=apparmor","name=seccomp,profile=builtin","name=cgroupns"]
ubuntu@VM-0-5-ubuntu:~$
```

4.3 Analisis Masalah dan Solusi (Troubleshooting)

Masalah yang sering muncul saat praktikum adalah *port conflict* saat menjalankan container, misalnya `error port is already allocated pada port 8080`. Cara mendiagnosisnya:

1. **Memeriksa Container Aktif:** Gunakan `docker ps` untuk melihat apakah ada container lama yang masih berjalan dan memakai port tersebut.
2. **Memeriksa Proses di Host:** Gunakan `sudo lsof -i :8080` untuk melihat aplikasi atau service lain di sistem host yang menduduki port.
3. **Solusi Penanganan:** Hentikan container lama dengan `docker rm -f <nama_container>` atau ubah konfigurasi mapping port pada host, misalnya menjadi `-p 8081:80`.

4.4 Analisis Keamanan dan Rekomendasi Produksi

Berdasarkan praktikum, terdapat beberapa risiko keamanan dan rekomendasi konfigurasi untuk lingkungan produksi:

1. **Port Binding Publik:** Penggunaan `-p 8080:80` secara default mengikat port ke `0.0.0.0` (terbuka ke jaringan publik). Di lingkungan produksi, port internal sebaiknya diikat ke `127.0.0.1` dan diakses melalui reverse proxy.

2. **User Non-Root di Container:** Proses di dalam container sebaiknya tidak dijalankan sebagai root (UID 0). Pada Dockerfile perlu ditambahkan instruksi USER non-root untuk membatasi hak akses jika container berhasil dieksploitasi.
3. **Pinning Versi Image:** Hindari penggunaan tag :latest pada production. Gunakan tag versi spesifik (misalnya nginx:1.26-alpine atau ubuntu:22.04) atau digest SHA256 agar deployment bersifat *reproducible* dan terhindar dari *breaking changes*.

4.5 Evaluasi dan Latihan Mandiri

1. Mengapa penggunaan tag latest tidak dianjurkan untuk deployment yang harus reproducible?

Tag latest bersifat dinamis dan selalu berubah mengikuti pembaruan terbaru dari image repository. Jika base image berubah, build di waktu yang berbeda dapat menghasilkan dependensi yang berbeda atau memicu kegagalan sistem yang tidak terduga. Untuk deployment yang konsisten (*reproducible*), sebaiknya gunakan tag versi spesifik (misalnya nginx:1.26-alpine) atau digest SHA256.

2. Jelaskan peran containerd dan runc dalam arsitektur Docker.

containerd bertindak sebagai pengelola *lifecycle* container tingkat tinggi (mengurus pengunduhan image, network, storage, dan pemantauan status container), sedangkan runc adalah komponen tingkat rendah yang bertugas langsung membuat dan menjalankan container sesuai standar OCI di atas kernel Linux.

3. Apa konsekuensi keamanan dari memasukkan user ke group docker?

Memasukkan user ke group docker memberikan hak akses penuh ke socket Docker (/var/run/docker.sock). Hal ini setara dengan memberikan akses root karena user dapat menjalankan container dengan hak istimewa tinggi atau melakukan mount root filesystem host tanpa perlu memasukkan password sudo.

4. Bandingkan layer image nginx:1.26-alpine dan image custom yang Anda buat.

Image nginx:1.26-alpine berisi layer dasar OS Alpine Linux dan instalasi Nginx default. Image custom pens-web:1.0 menggunakan image tersebut sebagai base layer, kemudian menambahkan satu layer baru dari instruksi COPY index.html yang mengganti halaman web default dengan file HTML praktikum.

5. Kapan sebaiknya memilih VM daripada container?

VM lebih tepat dipilih jika membutuhkan kernel atau sistem operasi yang berbeda dari host (misalnya menjalankan Windows di atas host Linux), memerlukan isolasi keamanan tingkat hardware yang sangat ketat (*multi-tenancy*), atau menjalankan aplikasi monolitik *legacy*. Container lebih tepat untuk aplikasi modern, *microservices*, dan pipeline CI/CD yang membutuhkan waktu mulai cepat dan konsumsi resource yang efisien.

BAB V KESIMPULAN

Containerization mempermudah pengelolaan dan distribusi aplikasi dengan memanfaatkan fitur kernel Linux (namespace dan cgroup) sehingga lebih ringan dan cepat dibandingkan Virtual Machine. Docker menyediakan arsitektur yang modular dan efisien berbasis image layer, di mana base image dapat digunakan bersama tanpa menduplikasi storage. Dalam praktik DevSecOps, keamanan container harus diperhatikan sejak awal, mulai dari pemilihan base image yang minimal, pembatasan hak akses user, pengelolaan port jaringan yang aman, hingga memastikan konfigurasi build dapat direplikasi dengan konsisten.

BAB VI DAFTAR PUSTAKA

1. NIST SP 800-190 Application Container Security Guide.
2. Open Container Initiative (OCI) Specifications.
3. Docker Engine Documentation and Best Practices.
4. DevOps Concept-Day2